CANlock — Système de Detection d'Intrusions sur Bus CAN

Guide complet du projet — Session H26

Équipe CIA – Universite Laval / Thales

Avril 2026

Résumé en 30 secondes

CANlock est un système de détection d'intrusions pour les véhicules connectés. Les véhicules modernes utilisent un réseau interne appelé **bus CAN** pour que leurs composants électroniques communiquent (freins, moteur, direction, etc.). Ce réseau n'a aucune sécurité native : un attaquant peut y injecter de faux messages pour prendre le contrôle du véhicule.
Notre solution : un modèle d'apprentissage profond (**CNN-LSTM Autoencoder**) qui apprend le comportement normal du réseau et détecte automatiquement les messages malveillants. Résultat : **99.5% des attaques détectees** avec seulement **6.2% de fausses alertes**.

Table des matières

1 Contexte : pourquoi ce projet ? 3
1.1 Le bus CAN en bref . . . 3
1.2 La menace . . . 3
1.3 Objectif de CANlock . . . 3

2 Les donnees 3
2.1 Source . . . 3
2.2 Structure d'un message CAN . . . 3
2.3 Le protocole J1939 . . . 4
2.4 Stockage . . . 4

3 Les attaques 4
3.1 DDoS (Denial of Service) . . . 4
3.2 Spoofing (Usurpation de payload) . . . 5
3.3 Replay (Rejeu) . . . 5
3.4 Suspension (Bus-Off) . . . 5
3.5 Masquerade (Usurpation d'identite) . . . 5
3.6 Parametres d'attaque . . . 5

4 Comment le modele detecte les attaques **6**
4.1 Le principe : detection par reconstruction 6
4.2 Les 11 features 6
4.3 Pre-traitement des donnees 6

5 Partitionnement des donnees **7**

6 Les deux architectures evaluees **7**
6.1 RNN-VAE (Variational Auto-Encoder avec LSTM) 7
6.2 CNN-LSTM Autoencoder (modele retenu) 7
6.3 Pourquoi le CNN-LSTM est meilleur 8

7 Entrainement **8**
7.1 Hyperparametres 8
7.2 Infrastructure et outils 8
7.3 Commande d'entrainement 9

8 Detection d'anomalies : du modele a la decision **9**
8.1 Calcul de l'erreur de reconstruction 9
8.2 Choix du seuil 9
8.3 Pourquoi le recall est prioritaire 9

9 Resultats **10**
9.1 Comparaison CNN-LSTM vs RNN-VAE 10
9.2 Matrices de confusion — CNN-LSTM 10
9.3 Matrices de confusion — RNN-VAE 10
9.4 Evolution des performances : comment on y est arrive 11

10 Architecture logicielle **11**
10.1 Structure du code 11
10.2 Services Docker 12
10.3 CLI : commandes disponibles 12

11 Conclusion **12**
11.1 Travaux futurs 12

1 Contexte : pourquoi ce projet ?

1.1 Le bus CAN en bref

Le **Controller Area Network (CAN)** est le protocole de communication standard dans l'industrie automobile depuis les années 1990. Dans un véhicule moderne, des dizaines d'unités électroniques (ECU) échangent en permanence des messages sur ce bus :

— L'ECU moteur envoie le régime et le couple
— L'ECU freinage envoie la pression sur les pédales
— Le tableau de bord lit ces messages pour afficher les informations
— Le système ABS réagit aux messages de freinage

Le problème fondamental : le protocole CAN a été conçu en 1986 pour la fiabilité, pas pour la sécurité. Il ne possède **aucun mécanisme d'authentification ni de chiffrement**. N'importe quel appareil connecté au bus peut envoyer n'importe quel message, et tous les ECU le recevront.

1.2 La menace

Un attaquant accédant au bus CAN (via le port OBD-II, un dongle Bluetooth compromis, ou le système d'infotainment) peut :

— **Injecter** de faux messages (ex : "freiner a fond")
— **Modifier** des messages en transit (ex : changer la vitesse affichee)
— **Supprimer** des messages (ex : empecher l'ABS de fonctionner)
— **Saturer** le bus pour bloquer toute communication

Ces attaques ont été demontrees dans la litterature sur des véhicules reels (Jeep Cherokee, Tesla, Toyota Prius).

1.3 Objectif de CANlock

Concevoir un **système de détection d'intrusions (IDS)** basé sur l'apprentissage profond, capable de détecter en temps réel les messages CAN malveillants avec :

— Un **recall** $\geq 99\%$ (ne rater presque aucune attaque)
— Un **taux de fausses alertes** faible (pour ne pas noyer l'operateur)

2 Les données

2.1 Source

Les données proviennent d'un jeu de données de bus CAN collecte depuis un **poids lourd de marque Renault**. Le jeu de données contient environ **490 millions d'enregistrements** respectant le protocole **CAN J1939** (standard pour véhicules lourds).

2.2 Structure d'un message CAN

Chaque message CAN (128 bits) est compose de :

— **Identifier** (29 bits) : identifie le type de message (PGN) et l'ECU source

— **DLC** (4 bits) : longueur du champ de données en octets
— **Data field** (0 a 64 bits) : les données utiles (le “payload”)

Exemple concret : un message avec l’identifiant 0x18FEF100 signifie “PGN 65265 (Vehicle Speed) envoyé par l’ECU a l’adresse 0x00”. Le payload contient la vitesse encodée sur quelques bits.

2.3 Le protocole J1939

J1939 définit une hiérarchie de décodage :

— **PGN** (Parameter Group Number) : identifie le groupe de paramètres (ex : “Electronic Engine Controller”)
— **SPN** (Suspect Parameter Number) : identifie un paramètre individuel dans le PGN (ex : “Engine Speed”, “Engine Torque”)
— Chaque SPN a une position en bits dans le payload, un facteur d’échelle et un offset pour convertir les bits bruts en valeur physique

2.4 Stockage

Les messages sont stockés dans une basé **PostgreSQL** (Docker, port 5435) avec le schéma :

```
Vehicle -> Session -> CanMessage (timestamp, can_identifier,
                                  length, payload)
PgnDefinition -> SpnDefinition (bit_start, bit_length,
                                scale, offset)
```

3 Les attaques

Pour tester notre IDS, nous generons des **attaques synthetiques réalistes** a partir de trafic CAN reel. Cinq types d’attaques couvrent les principales menaces documentees dans la litterature.

3.1 DDoS (Denial of Service)

Objectif : saturer le bus CAN pour empecher les communications normales.
Comment : envoi massif de messages sur un meme identifiant (3000 repetitions, intervalle de 0.0001s).
Effet : les messages légitimes sont retardes ou perdus, le véhicule peut devenir incontrolable.

3.2 Spoofing (Usurpation de payload)

Objectif : injecter de fausses données en usurpant un ECU légitime.
Comment : pour 10% des messages, modifie les octets du payload en appliquant un bruit gaussien.
Effet : le véhicule recoit des valeurs de capteur erronees (ex : fausse vitesse, faux couple moteur).

3.3 Replay (Rejeu)

Objectif : capturer puis réémettre des messages légitimes a un autre moment.
Comment : 10% des messages sont copiés et réinjectés avec 1 seconde de délai.
Effet : force une action passee (ex : rejeu d'une commande de deverrouillage).

3.4 Suspension (Bus-Off)

Objectif : deconnecter un ECU du réseau.
Comment : génère des erreurs ciblées qui incrémentent le compteur d'erreurs de l'ECU victime (+8 par erreur). Au delà de 256, l'ECU se met en Bus-Off (mécanisme natif CAN).
Effet : l'ECU ciblé est isolé, ses messages disparaissent du bus.

3.5 Masquerade (Usurpation d'identité)

Objectif : usurper l'identité d'un ECU pour prendre le contrôle de ses fonctions.
Comment : modifie l'adresse source (LSB) de l'identifiant CAN (8% des messages).
Effet : l'attaquant parle "au nom" d'un ECU légitime (ex : se faire passer pour l'ECU moteur).

3.6 Paramètres d'attaque

Les taux d'injection par défaut (2–3%) étaient trop faibles : les séquences "attaquées" contenaient majoritairement du trafic normal, rendant l'apprentissage inefficace. Les paramètres optimaux :

Attaque	Paramètre	Defaut	Optimal
Spoofing	injection_rate	3%	**10%**
Masquerade	prob	2%	**8%**
Replay	replay_rate	3%	**10%**
Suspension	suspend_fraction	2%	**8%**
DDoS	repetitions	1000	**3000**

Table 1 – Paramètres d'attaque : valeurs par défaut vs optimales

4 Comment le modèle détecte les attaques

4.1 Le principe : détection par reconstruction

Notre approche est **non supervisee** : le modèle n'a **jamais vu d'attaque** pendant l'entraînément. Il apprend uniquement a reconstruire du trafic CAN normal.

1. **Entraînement :** on donne au modèle des milliers de séquences de trafic normal. Il apprend a les compresser puis les reconstruire fidèlement.
2. **Detection :** face a un nouveau message, le modèle tente de le reconstruire. Si l'erreur de reconstruction est **élevée**, le message est "hors distribution" → probablement une **anomalie**.

Analogie : comme un employe de banque qui voit des milliers de signatures normales chaque jour. Sans avoir jamais vu de faux, il détecte quand une signature "ne ressemble pas" a ce qu'il connait.

4.2 Les 11 features

Plutot que de décoder les messages CAN en valeurs physiques (SPN), ce qui est lent et perd de l'information, nous extrayons **11 features directement des messages bruts** :

Feature	Description	Pourquoi ?
`can_id`	Identifiant CAN	Change lors d'une masquerade ou DDoS
`byte_0..7`	8 octets du payload	Changes lors d'un spoofing
`delta_t`	Temps depuis le msg précédent	Anormal lors d'un replay (délai), suspension (gap) ou DDoS (trop rapide)
`freq`	Fréquence de cet ID dans les 50 derniers msgs	Explose lors d'un DDoS, chute lors d'une suspension

Table 2 – Les 11 features et ce qu'elles détectent

Pourquoi pas le décodage SPN ? L'approche initiale décodait chaque message en valeurs SPN via la basé J1939. Problèmes :

— Tres lent (∼15 min pour 40k messages vs quelques secondes avec les payloads bruts)
— Le forward-fill et la normalisation lissaient les modifications des attaques
— Résultat : ROC-AUC de 0.52 (= hasard)

4.3 Pre-traitement des données

1. **Clipping des outliers** au 99e percentile sur `delta_t` et `freq` (evite les spikes d'entraînément)
2. **Normalisation** StandardScaler (fit uniquement sur les données normales)
3. **Séquences temporelles** : fenêtre de 50 messages, stride de 25 (chevauchement)

5 Partitionnement des données

Le partitionnement est **crucial** pour la fiabilité des résultats. Deux règles :

Règles de partitionnement

1. **Split temporel** (pas de shuffle) : les données CAN sont des series temporelles. Melanger passe et futur causerait du *data leakage*.
2. **Attaques uniquement dans le test** : le modèle ne voit **jamais** d'attaque pendant l'entraînément ni la validation.

Schema concret :

— 500 000 messages charges depuis la BD, tries par timestamp
— 80% premiers → **train** + **val** (uniquement normaux)
— 20% derniers → **test** (on y applique les 5 attaques)
— Les séquences normales sont splittees : **75% train / 15% val / 10% test**
— Le test final = 10% normales + toutes les attaques → permet de mesurer FP et TP

Avec 500k messages, stride=25, seq_len=50 : environ **12 000 séquences d'entraînément**, **2 400 de validation**, **5 768 de test** (1 601 normales + 4 167 attaquées).

6 Les deux architectures évaluées

Les deux modèles sont des **autoencodeurs** : ils compriment l'entrée en une représentation compacte ("bottleneck") puis tentent de reconstruire l'entrée originale. La différence est dans comment ils compriment.

6.1 RNN-VAE (Variational Auto-Encoder avec LSTM)

Idée : apprendre une *distribution* du trafic normal, pas juste une compression.
Architecture :

— **Encodeur :** LSTM bidirectionnel (64 hidden) → Dropout 0.2 → deux couches linéaires qui produisent μ et $\log \sigma^2$ (32 dimensions chacune)
— **Espace latent :** $z = \mu + \sigma \cdot \epsilon$ avec $\epsilon \sim \mathcal{N}(0, I)$ (reparameterization trick)
— **Décodeur :** linéaire 32→128 → repeat sur la longueur de sequence → LSTM bidirectionnel → Dropout → linéaire → 11 features

Loss :

$$\mathcal{L} = \underbrace{\mathrm{MSE}(x, \hat{x})}_{\text{reconstruction}} + \underbrace{10^{-3} \cdot D_{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, I))}_{\text{régularisation latente}} \tag{1}$$

152K paramètres entrainables.

6.2 CNN-LSTM Autoencoder (modèle retenu)

Idée : utiliser des convolutions pour capter les patterns *locaux* dans le payload, et un LSTM pour les dépendances *temporelles*.
Architecture :

— **Encodeur CNN :** Conv1d(11→32) + BN + ReLU + Pool(/2) → Conv1d(32→64) + BN + ReLU + Pool(/2)

— **Encodeur LSTM :** LSTM bidirectionnel (32 hidden) + Dropout 0.2
— **Bottleneck :** linéaire 64→32
— **Décodeur LSTM :** LSTM bidirectionnel (64 hidden)
— **Décodeur CNN :** ConvTranspose1d(128→64) + BN + ReLU → ConvTranspose1d(64→32) + BN + ReLU → Conv1d(32→11) + interpolation

Loss :

$$\mathcal{L} = \mathrm{MSE}(x, \hat{x}) \tag{2}$$

106K paramètres entrainables.

6.3 Pourquoi le CNN-LSTM est meilleur

Aspect	RNN-VAE	CNN-LSTM
Patterns locaux (spoofing)	LSTM seul	**CNN** (convolutions)
Patterns temporels (DDoS, replay)	LSTM	**LSTM**
Stabilite entraînément	Trade-off MSE/KL	**MSE seul**
Fausses alertes (seuil recall 99.5%)	29.4%	**6.2%**

Table 3 – Pourquoi le CNN-LSTM surpasse le RNN-VAE

7 Entraînement

7.1 Hyperparamètres

Les deux modèles ont été entraînés avec les **memes hyperparamètres** pour une comparaison équitable :

Hyperparamètre	Valeur
Messages charges	500 000
Batch size	128
Learning rate	5×10^{-4}
Epochs max	200
Sequence length	50
Stride	25
Early stopping patience	20 epochs
Gradient clipping	1.0
Optimizer	Adam
Scheduler	ReduceLROnPlateau (factor=0.5, patience=5)
Seed	42

Table 4 – Hyperparamètres d'entraînément (identiques pour les deux modèles)

7.2 Infrastructure et outils

— **Matériel :** Intel Core 7 @ 1.8 GHz, CPU uniquement (pas de GPU)
— **Framework :** PyTorch Lightning

— **Suivi d'expériences :** MLflow (`http://localhost:5050`)
— **Base de données :** PostgreSQL (Docker, port 5435)
— **Gestion des dépendances :** uv + pyproject.toml
— **Temps d'entraînément :** ∼30–45 min par modèle (CPU)

7.3 Commande d'entraînément

```
uv run train-models --model-type cnn_lstm \
    --limit 500000 --epochs 200 --batch-size 128 \
    --stride 25 --seq-len 50 --lr 5e-4 \
    --spoof-injection-rate 0.10 --masq-prob 0.08 \
    --replay-rate 0.10 --susp-suspend-fraction 0.08 \
    --ddos-repetitions 3000 --target-recall 0.995
```

8 Detection d'anomalies : du modèle a la decision

8.1 Calcul de l'erreur de reconstruction

Pour chaque sequence de test, on calcule l'erreur MSE entre l'entrée et la sortie du modèle :

$$e_i = \frac{1}{T \times F} \sum_{t=1}^{T} \sum_{f=1}^{F} (x_{i,t,f} - \hat{x}_{i,t,f})^2 \tag{3}$$

ou $T = 50$ (longueur de sequence) et $F = 11$ (nombre de features).

Intuition : le modèle reconstruit bien ce qu'il connait (trafic normal → erreur faible). Face a une attaque, la reconstruction est mauvaise → erreur élevée.

8.2 Choix du seuil

L'erreur de reconstruction seule ne suffit pas : il faut un **seuil** au-dessus duquel on declare "anomalie". Deux stratégies :

1. **Seuil F1-optimal :** maximise le F1-score (équilibre precision/recall). Bon pour un usage general.
2. **Seuil recall-ciblé :** garantit un recall minimum (ex : 99.5%). Indispensable pour la sécurité vehiculaire : mieux vaut quelques fausses alertes qu'une attaque ratée.

8.3 Pourquoi le recall est prioritaire

— **Faux négatif** (attaque ratée) → conséquence potentiellement mortelle (prise de contrôle du véhicule)
— **Faux positif** (fausse alerte) → génère un log/alerte, mais pas de danger
— Sur un bus CAN avec des centaines de milliers de messages/minute, meme un FPR de 6% génère beaucoup d'alertes → filtrage en aval nécessaire

9 Résultats

9.1 Comparaison CNN-LSTM vs RNN-VAE

Résultats avec le seuil recall $\geq$ 99.5% :

Métrique	**RNN-VAE**	**CNN-LSTM**
ROC-AUC	0.990	**0.998**
FPR (fausses alertes)	29.4%	**6.2%**
Recall (détection)	99.5%	99.5%
F1-Score	0.940	**0.988**
Accuracy	91.0%	**98.0%**

TABLE 5 – Comparaison des deux modèles (seuil recall $\geq$ 99.5%)

Lecture : les deux modèles détectent 99.5% des attaques, mais le CNN-LSTM génère **5 fois moins de fausses alertes** (6.2% vs 29.4%).

9.2 Matrices de confusion — CNN-LSTM

	Seuil F1-optimal		**Seuil Recall $\geq$ 99.5%**	
	Pred. Normal	Pred. Anomalie	Pred. Normal	Pred. Anomalie
Reel Normal (1601)	1559	42	1502	99
Reel Anomalie (4167)	45	4122	21	4146
Recall	98.9%		**99.5%**	
FPR	**2.6%**		6.2%	

TABLE 6 – Matrices de confusion du CNN-LSTM

Lecture : avec le seuil recall $\geq$ 99.5%, le modèle ne rate que **21 attaques sur 4167**. Il génère 99 fausses alertes sur 1601 séquences normales.

9.3 Matrices de confusion — RNN-VAE

	Seuil F1-optimal		**Seuil Recall $\geq$ 99.5%**	
	Pred. Normal	Pred. Anomalie	Pred. Normal	Pred. Anomalie
Reel Normal (1601)	1486	115	1131	470
Reel Anomalie (4167)	134	4033	21	4146
Recall	96.8%		99.5%	
FPR	7.2%		29.4%	

TABLE 7 – Matrices de confusion du RNN-VAE

Lecture : le RNN-VAE atteint le meme recall, mais génère **470 fausses alertes** (vs 99 pour le CNN-LSTM).

9.4 Evolution des performances : comment on y est arrive

L'optimisation a été itérative. Chaque étape a apporte une amélioration mesurable :

Étape	ROC-AUC	FPR	Recall	Ce qui a change
Départ (SPN, 50k)	0.52	1.00	0.997	Décodage SPN, tout classe anomalie
Payloads bruts	0.54	1.00	0.997	8 octets seulement, pas assez
+ CAN ID + temps	0.67	0.99	0.997	11 features, debut de discrimination
+ 200k messages	0.70	0.68	0.917	Plus de données d'entraînément
+ 500k, stride=10	0.92	0.29	0.917	Beaucoup plus de séquences
+ Taux attaque x3	0.996	0.037	0.979	Attaques plus visibles
Config finale	**0.998**	**0.062**	**0.995**	Epochs 200, patience 20

TABLE 8 – Evolution des performances du CNN-LSTM (chaque ligne = une experience)

10 Architecture logicielle

10.1 Structure du code

```
src/canlock/
    attacks/              # Generation d'attaques
        attack_base.py    #   Classe abstraite + utilitaires bit-level
        attack_ddos.py    #   Attaque DDoS
        spoofing_attack.py#   Spoofing (bruit gaussien)
        replay_attack.py  #   Replay (rejeu avec délai)
        suspension_attack.py # Suspension (Bus-Off)
        masquerade_attack.py # Masquerade (usurpation ID)
    data/
        can_data_module.py #  DataModule PyTorch Lightning
    db/
        models.py         #  ORM SQLModel (Vehicle, Session,
                          #    CanMessage, PGN, SPN)
        database.py       #  Connexion PostgreSQL
    models/
        cnn_lstm_autoencoder.py  # CNN-LSTM (modèle retenu)
        rnn_vae.py               # RNN-VAE
        anomaly_detector.py      # Évaluation (seuil, métriques)
    training/
        train_models.py   #  Pipeline d'entraînément complet (CLI)
    décoder.py            #  Décodeur SPN/PGN J1939
    cli.py                #  Points d'entrée CLI

scripts/
    attack_synthesis.py   # Generation de datasets attaques
    attack_inspect.py     # Inspection/validation
```

```
    attack_visualize.py   # Visualisation
    generate_poster_figures.py     # Figures pour l'affiche
    generate_architecture_figures.py # Diagrammes d'architecture

notebooks/
    rnn_vae_model.ipynb    # Expérimentation RNN-VAE
    cnn_lstm_model.ipynb   # Expérimentation CNN-LSTM
    can_data_analysis.ipynb # Analyse exploratoire
```

10.2 Services Docker

```
# docker-compose.yml
services:
  postgres-backend:  # BDD CAN (port 5435)
  mlflow:            # Suivi d'exp riences (port 5050)
```

10.3 CLI : commandes disponibles

```
uv run train-models     # Entra nement des mod les
uv run download-heavy-truck-data  # Telechargement donn es
uv run analyze-session  # Analyse d'une session CAN
```

11 Conclusion

Résultats cles

- Le **CNN-LSTM Autoencoder** détecte **99.5% des intrusions** avec **6.2% de fausses alertes**
- ROC-AUC de **0.998** (quasi-parfait)
- Detecte les **5 types d'attaques** : DDoS, spoofing, masquerade, replay, suspension
- Entraînement **non supervise** (pas besoin d'exemples d'attaques pour entraîner)
- **11 features** extraites des messages bruts (pas de décodage SPN couteux)
- Surpasse le RNN-VAE : **5x moins de fausses alertes** pour le meme recall

11.1 Travaux futurs

- Validation sur des **attaques réelles** (pas uniquement synthetiques)
- Optimisation pour **inférence en temps reel** sur hardware embarqué (microcontrôleur)
- Ajout d'attaques plus sophistiquées (fuzzing, attaques graduelles, attaques zero-day)
- Évaluation sur d'autres jeux de données CAN (véhicules légers, protocole CAN-FD)
- Reduction du FPR par filtrage contextuel post-détection

Références

[1] Rajapaksha, S. et al. (2023). Beyond vanilla : Improved autoencoder-based ensemble in-vehicle IDS. *J. Information Security and Applications*, 77.

[2] Balaji, P. et al. (2022). CANLite : Anomaly Detection in CAN with Multitask Learning. *IEEE VTC2022-Spring*.

[3] Alkhatib, N. et al. (2022). CAN-BERT do it ? CAN IDS based on BERT. *IEEE AICCSA*.

[4] Gao, F. et al. (2024). Multi-Attack IDS for In-Vehicle CAN-FD Messages. *Sensors*, 24(11).

[5] Hoang, T.-N. & Kim, D. (2022). Semi-supervised convolutional adversarial autoencoders for in-vehicle intrusion détection. *Vehicular Communications*, 38.

[6] Mansourian, P. et al. (2023). Deep Learning-Based Anomaly Detection for CAVs Using Spatiotemporal Information. *IEEE TITS*, 24(12).

[7] Xiao, J. et al. (2023). Robust anomaly-based IDS for in-vehicle network by graph neural network framework. *Applied Intelligence*, 53(3).

[8] Cao, J. et al. (2024). Anomaly Detection for In-Vehicle Network Using Self-Supervised Learning. *IEEE TITS*, 25(7).